

PyTorch Training Pipeline

A Manual Binary Classifier from Data to Evaluation

Abstract

A neural-network training pipeline is a repeated sequence of prediction, error measurement, differentiation, parameter update, and gradient clearing. This chapter constructs that sequence manually for a one-neuron breast-cancer classifier. The manual implementation makes every state transition visible before higher-level abstractions such as `nn.Module`, built-in losses, and optimizers replace the hand-written components.

Contents

1	The objective and the complete workflow	1
2	Data preparation	2
2.1	Cleaning, splitting, scaling, and encoding	2
2.2	NumPy arrays to tensors	4
3	A one-neuron model built by hand	4
4	Binary cross-entropy	6
5	Training by full-batch gradient descent	7
5.1	Model creation and parameter inspection	7
5.2	Epoch loop	7
5.3	Why <code>no_grad()</code> and <code>zero_()</code> are separate	8
6	Evaluation	9
7	Manual components and their idiomatic replacements	10
8	Key takeaways	10
9	Review questions	11

1 The objective and the complete workflow

The aim is to understand the mechanics of training rather than to optimize medical-classification performance. A single sigmoid neuron receives 30 numeric features and predicts one of two classes. Despite its simplicity, the workflow is structurally identical to that used by deeper models:

1. prepare train and test tensors;

2. initialize trainable weights and bias;
3. run a forward pass;
4. compute a scalar loss;
5. run backpropagation;
6. update parameters without recording the update in the graph;
7. clear accumulated gradients;
8. repeat for several epochs, then evaluate on held-out data.

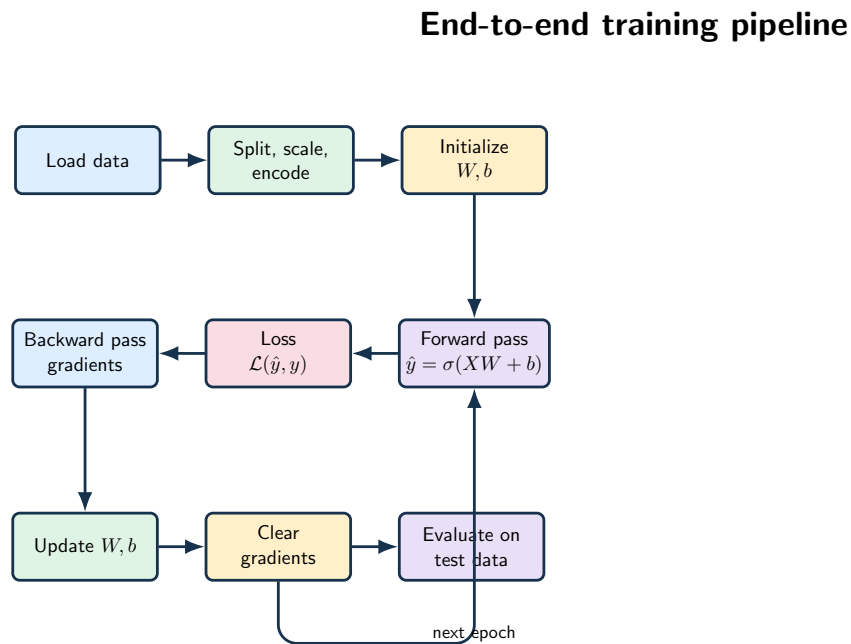


Figure 1: The complete manual training workflow. The feedback path returns to the forward pass only after the old gradients are cleared. [Technical reference](#).

2 Data preparation

The demonstrated dataset is the Wisconsin breast-cancer table: 569 patient records, an identifier, a diagnosis label, 30 numeric measurements, and an empty trailing column. The prediction target is binary: malignant or benign.

2.1 Cleaning, splitting, scaling, and encoding

The exact external CSV address was not recoverable from the accessible source material, so the reconstruction keeps it as an explicit placeholder. This avoids inventing an official URL while preserving every demonstrated operation.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

DATA_URL = "<official breast-cancer CSV URL from the notebook>"
df = pd.read_csv(DATA_URL)

# The source table contains 569 rows and 33 columns before cleaning.
df.drop(columns=["id", "Unnamed: 32"], inplace=True)

X = df.drop(columns=["diagnosis"])
y = df["diagnosis"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

encoder = LabelEncoder()
y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)

```

Listing 1: Load, clean, split, scale, and encode the dataset

Meaning of each block.

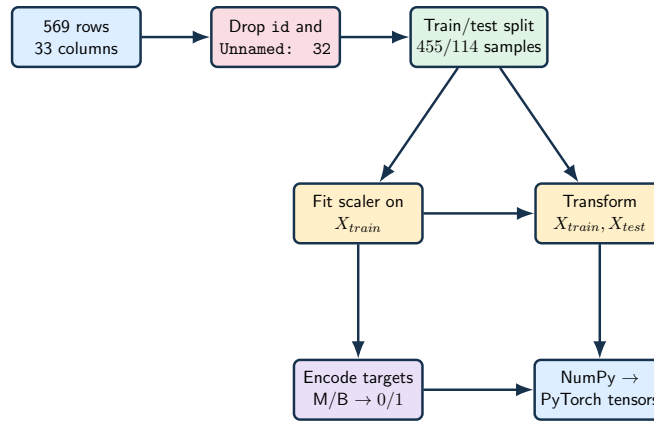
- Dropping `id` prevents an arbitrary record identifier from acting as a feature. Dropping `Unnamed: 32` removes the empty column.
- A 20% test split produces about 455 training rows and 114 test rows. Training data changes the parameters; test data estimates generalization.
- `fit_transform` learns the mean and standard deviation from *training data only*. `transform` applies those same statistics to test data, preventing test information from leaking into training.
- Label encoding maps the two textual diagnoses to integers. The exact mapping must be checked with `encoder.classes_`; the mathematics only requires consistent 0/1 targets.

For feature j , standardization computes

$$x'_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

Here x_{ij} is feature j of sample i , μ_j is the training-set mean of feature j , s_j is its training-set standard deviation, and x'_{ij} is the scaled value. Centering and scaling keep large-magnitude measurements from dominating the dot product merely because of their units.

Data preparation and leakage-safe scaling



Only training statistics should determine the scaler. The target must be numeric, and feature/parameter dtypes must match.

Figure 2: Leakage-safe preprocessing: split first, fit transformations on training data, and reuse them on test data. [Technical reference](#).

2.2 NumPy arrays to tensors

```
import torch

X_train_tensor = torch.from_numpy(X_train)
X_test_tensor = torch.from_numpy(X_test)
y_train_tensor = torch.from_numpy(y_train).reshape(-1, 1).to(torch.float64)
y_test_tensor = torch.from_numpy(y_test).reshape(-1, 1).to(torch.float64)

print(X_train_tensor.shape, X_train_tensor.dtype)
print(y_train_tensor.shape, y_train_tensor.dtype)
```

Listing 2: Convert prepared arrays to dtype-compatible PyTorch tensors

A typical shape check reports `torch.Size([455, 30])` for the training features and `torch.Size([455, 1])` for the labels. `StandardScaler` normally returns NumPy `float64`, so the model parameters below also use `torch.float64`. Matrix multiplication requires compatible dtypes.

Intuition

A tensor is not merely a container. Tensors participating in a differentiable computation carry graph information, while parameters with `requires_grad=True` accumulate derivatives in their `.grad` attributes.

3 A one-neuron model built by hand

For a batch $X \in \mathbb{R}^{m \times d}$, define

$$z = XW + b, \quad \hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The symbol m is the number of samples, $d = 30$ is the number of features, $W \in \mathbb{R}^{30 \times 1}$ is the weight column, $b \in \mathbb{R}$ is the scalar bias, $z \in \mathbb{R}^{m \times 1}$ is the logit column, and $\hat{y} \in (0, 1)^{m \times 1}$ contains predicted probabilities. The sigmoid maps any real logit to a value between zero and one.

```

class MySimpleNN:
    def __init__(self, X):
        self.weights = torch.rand(
            X.shape[1], 1,
            dtype=torch.float64,
            requires_grad=True
        )
        self.bias = torch.zeros(
            1,
            dtype=torch.float64,
            requires_grad=True
        )

    def forward(self, X):
        z = torch.matmul(X, self.weights) + self.bias
        y_pred = torch.sigmoid(z)
        return y_pred

    def loss_function(self, y_pred, y):
        epsilon = 1e-7
        y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
        sample_losses = -(
            y * torch.log(y_pred)
            + (1 - y) * torch.log(1 - y_pred)
        )
        return sample_losses.mean()

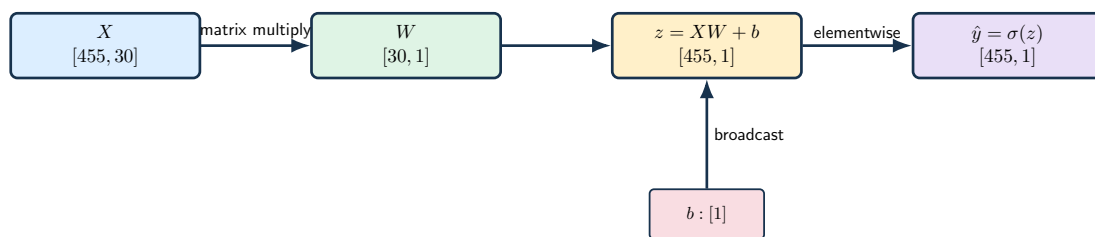
```

Listing 3: Manual single-neuron model and binary cross-entropy

Constructor state. `X.shape[1]` discovers the feature count rather than hard-coding 30. Random weights break symmetry; the single bias starts at zero. Both tensors are leaves requiring gradients, so backpropagation writes derivatives into `weights.grad` and `bias.grad`.

Forward state. Multiplying $[m, 30]$ by $[30, 1]$ produces $[m, 1]$. Adding a one-element bias uses broadcasting. Sigmoid then acts element by element without changing shape.

Single-neuron tensor-shape flow



Thirty input features require thirty weights. One output neuron contributes one bias and one probability per sample.

Figure 3: Tensor shapes through the one-neuron forward pass. [Technical reference](#).

Computational graph and gradient paths

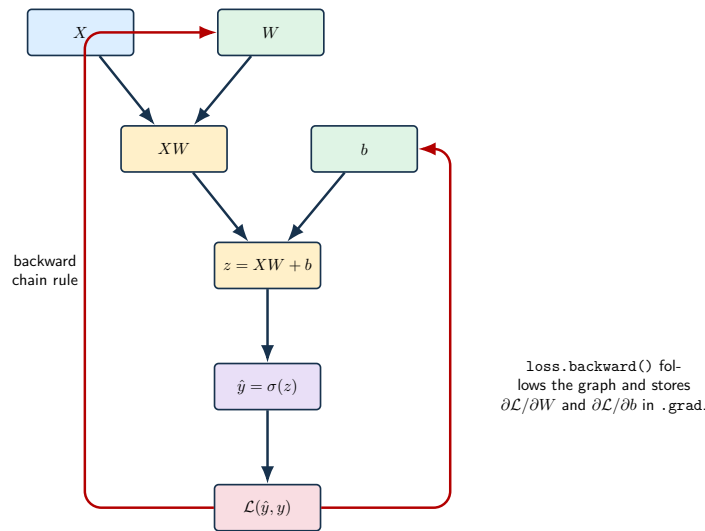


Figure 4: The differentiable graph links the scalar loss back to both trainable leaves. Backward arrows are routed outside the forward path. [Technical reference](#).

4 Binary cross-entropy

For m samples, binary cross-entropy is

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

For sample i , $y_i \in \{0, 1\}$ is the true class and $\hat{y}_i \in (0, 1)$ is the predicted probability of class 1. When $y_i = 1$, only $-\log \hat{y}_i$ remains; when $y_i = 0$, only $-\log(1 - \hat{y}_i)$ remains. The average produces one scalar whose gradient summarizes the batch. Clamping by a small ϵ prevents numerical evaluation of $\log 0$.

Binary cross-entropy for one sample

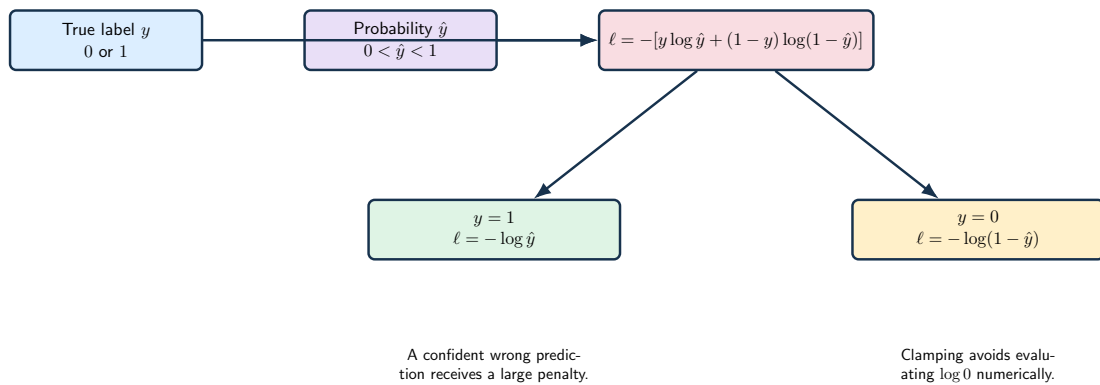


Figure 5: Binary cross-entropy selects the positive-class or negative-class penalty according to the target. [Technical reference](#).

5 Training by full-batch gradient descent

5.1 Model creation and parameter inspection

```
model = MySimpleNN(X_train_tensor)
print(model.weights.shape)  # torch.Size([30, 1])
print(model.bias.shape)    # torch.Size([1])
print(model.weights.requires_grad, model.bias.requires_grad)
# True True
```

Listing 4: Instantiate the model and verify parameter shapes

The values in `weights` differ from run to run because initialization is random. The shapes and gradient flags are deterministic.

5.2 Epoch loop

```
learning_rate = 0.1
epochs = 25

for epoch in range(epochs):
    # 1. Forward pass: one probability for every training sample.
    y_pred = model.forward(X_train_tensor)

    # 2. Scalar batch loss.
    loss = model.loss_function(y_pred, y_train_tensor)

    # 3. Backward pass: populate weights.grad and bias.grad.
    loss.backward()

    # 4. Gradient-descent update, excluded from the graph.
    with torch.no_grad():
        model.weights -= learning_rate * model.weights.grad
        model.bias -= learning_rate * model.bias.grad

    # 5. PyTorch accumulates gradients, so clear both leaves.
    model.weights.grad.zero_()
    model.bias.grad.zero_()

    print(f"Epoch {epoch + 1:02d}: loss={loss.item():.4f}")
```

Listing 5: Manual forward, backward, update, and gradient-clearing loop

The update equations are

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}_t, \quad b_{t+1} = b_t - \eta \nabla_b \mathcal{L}_t.$$

Here t is the epoch index, $\eta > 0$ is the learning rate, $\nabla_W \mathcal{L}_t$ is the loss derivative with respect to every weight, and $\nabla_b \mathcal{L}_t$ is the derivative with respect to the bias. Subtracting the gradient moves parameters locally toward lower loss; η controls the step size.

Expected behavior. The first loss depends on random initialization and was observed around 3 in the demonstration. Across 25 epochs it generally decreases. Exact numbers are not deterministic unless a random seed, identical preprocessing, and identical data split are fixed.

One epoch: forward, backward, update, clear

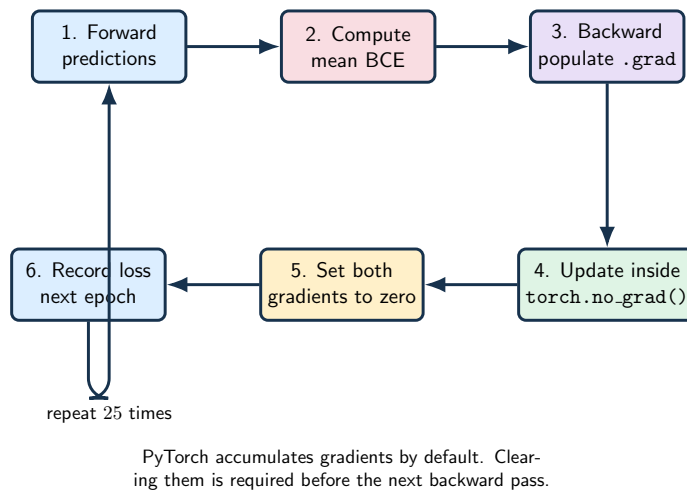


Figure 6: The six state transitions within one epoch. [Technical reference](#).

Parameter update and graph boundary

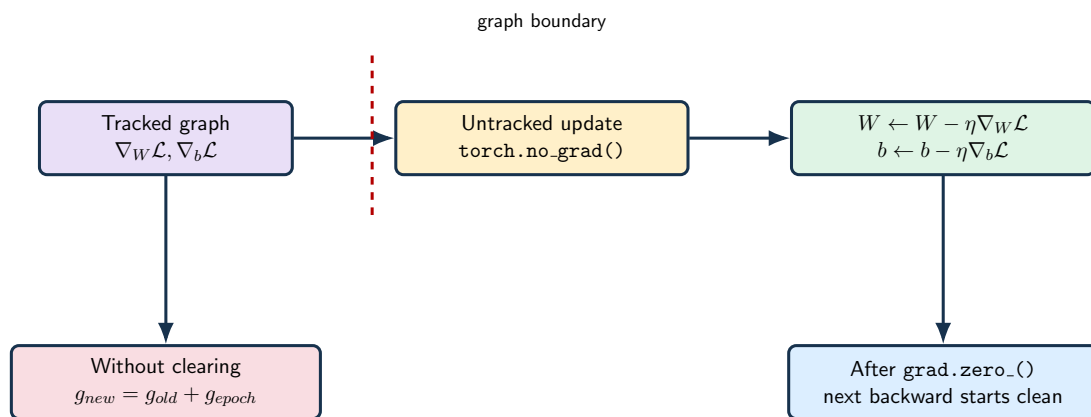


Figure 7: Differentiation is tracked, but the parameter mutation is deliberately outside the graph. [Technical reference](#).

5.3 Why `no_grad()` and `zero_()` are separate

`torch.no_grad()` prevents the update itself from becoming another differentiable operation. It does *not* erase existing gradients. Conversely, `grad.zero_()` resets stored derivative values but does not disable graph recording. Both operations are needed in the manual loop.

Pitfall

Calling `loss.backward()` repeatedly without clearing gradients performs accumulation:

$$g_{\text{stored}} \leftarrow g_{\text{stored}} + g_{\text{new}}.$$

Accumulation is useful for specialized techniques, but it is not the intended behavior of this full-batch example.

6 Evaluation

Evaluation uses the learned parameters but does not update them. Probabilities must be converted to class labels before comparison with 0/1 targets.

```
with torch.no_grad():
    y_prob = model.forward(X_test_tensor)

threshold = 0.5
y_pred = (y_prob >= threshold).to(torch.float64)
accuracy = (y_pred == y_test_tensor).to(torch.float64).mean()

print(y_prob.shape)           # torch.Size([114, 1]) for a 20% split
print(torch.unique(y_pred))    # tensor([0., 1.], dtype=torch.float64)
print(f"Accuracy: {accuracy.item() * 100:.2f}%")
```

Listing 6: Evaluate probabilities, threshold classes, and compute accuracy manually

The indicator rule is

$$\hat{c}_i = \mathbf{1}[\hat{y}_i \geq \tau],$$

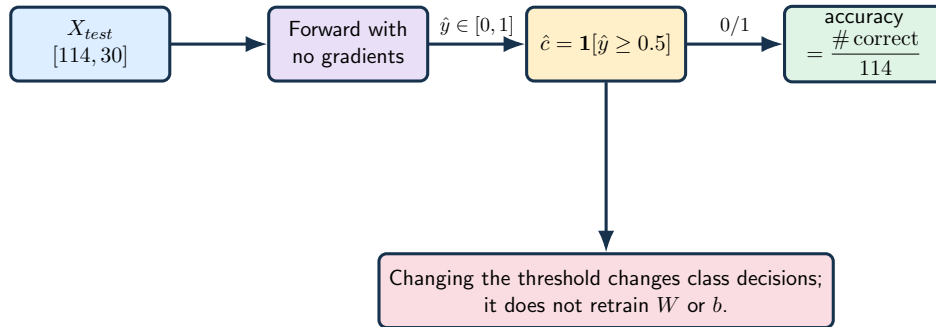
where $\hat{c}_i$ is the predicted class, $\hat{y}_i$ is the probability, and $\tau = 0.5$ is the chosen threshold. Accuracy is

$$\text{accuracy} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbf{1}[\hat{c}_i = y_i].$$

The numerator counts correct predictions and n_{test} is the number of test samples. The demonstration obtained roughly 51%, described as poor; random initialization and implementation details make the precise value variable. Changing τ may change decisions, but it does not improve the learned parameters.

For medical classification, accuracy alone can hide clinically important errors. Precision measures how many predicted positives are truly positive, recall measures how many true positives are found, and F1 balances precision and recall.

Evaluation and thresholding



Accuracy is only one metric. Precision, recall, and F1 are often important for medical classification.

Figure 8: Evaluation converts probabilities to decisions and compares them with held-out labels. [Technical reference](#).

7 Manual components and their idiomatic replacements

Concern	Manual implementation here	Idiomatic PyTorch replacement
Model parameters	Create tensors and set <code>requires_grad=True</code>	Subclass <code>nn.Module</code> ; use <code>nn.Linear</code>
Forward computation	Call a custom <code>forward</code> method explicitly	Invoke <code>model(X)</code> ; module dispatches to <code>forward</code>
Loss	Hand-code mean binary cross-entropy	<code>nn.BCELoss</code> or preferably <code>nn.BCEWithLogitsLoss</code>
Update	Subtract gradients inside <code>no_grad</code>	<code>optimizer.step()</code>
Gradient reset	Clear each parameter manually	<code>optimizer.zero_grad()</code>
Optimization algorithm	Plain gradient descent	SGD, Adam, RMSprop, and others in <code>torch.optim</code>

Table 1: The manual pipeline exposes mechanics that higher-level PyTorch APIs automate.

The higher-level version does not alter the conceptual order: forward, loss, backward, update, clear. It reduces bookkeeping and automatically registers parameters so that serialization, device movement, and optimizer integration work reliably.

8 Key takeaways

1. A training pipeline is a state machine: predictions create a graph, loss closes it to a scalar, backward fills gradient buffers, the optimizer changes parameters, and gradient clearing prepares the next iteration.
2. Shapes encode model structure. Thirty input features and one output neuron imply $W \in \mathbb{R}^{30 \times 1}$ and one bias.

3. Dtype compatibility is essential; the demonstrated scaled arrays and manual parameters use `float64`.
4. Binary cross-entropy rewards probabilities aligned with binary targets and strongly penalizes confident errors.
5. Parameter mutation belongs outside autograd tracking, whereas forward and loss computations belong inside it.
6. Test evaluation disables gradients and must remain separate from training.
7. Poor accuracy does not invalidate the workflow example; it indicates that model capacity, optimization choices, preprocessing, thresholding, or evaluation design need further work.

9 Review questions

1. Why does $X[455, 30]W[30, 1]$ produce one prediction per training sample?
2. What would happen if the test set were used to fit the scaler?
3. Why does `loss.backward()` need a scalar loss in this example?
4. Why is the weight update wrapped in `torch.no_grad()`?
5. How would the second epoch's gradient change if `zero_()` were omitted?
6. Why can changing a classification threshold alter accuracy without changing the model?